

# Алгоритми компресије података

Марко Вучић  
yellow.marko@gmail.com

26.4.2026.

# Основе теорије информација

## 1.1 Увод

Замислимо да треба да запишемо информацију о побједнику неког шаховског турнира, на којем учествују 4 такмичара, од којих неки имају веће шансе за побједу, неки мање. Циљ је да то урадимо кориштењем што мање ресурса.

Приступ који се чини као најочигледнији је да сваком такмичару додијелимо бинарни код дужине 2, тј. редом 00, 01, 10, 11. Овим смо, у случају било чије побједе, резултат записали са 2 бита.

Међутим, можемо и боље од овога! Наиме, најјачи такмичар има највећу вјероватноћу да побиједи, док је шанса да онај најслабији побиједи најмања. Ако додијелимо краћи код оном најјачем, а при том компензујемо са дужим кодом за оног најслабијег - у просјеку можемо добити бољи резултат. Нека је вјероватноћа побједе играча с датим редним бројем дата расподелом:

$$X \sim \begin{pmatrix} 1 & 2 & 3 & 4 \\ \frac{1}{2} & \frac{1}{4} & \frac{1}{8} & \frac{1}{8} \end{pmatrix}$$

Тада, ако додијелимо првом играчу код 0, другом играчу код 10, тре-

ћем 110, а четвртом 111,<sup>1</sup> можемо пронаћи да је очекивана вриједност дужине кода:

$$E[L] = \sum_{i=1}^4 p(i)L(i) = \frac{1}{2} \cdot 1 + \frac{1}{4} \cdot 2 + \frac{1}{8} \cdot 3 + \frac{1}{8} \cdot 3 = \frac{7}{4}$$

Дакле, успјели смо овим смањити просјечну дужину кода за 12.5%.

Није нам ни потребан овакав статистички поглед са релативно апстрактним појмом очекивања - можемо разматрати ситуацију у којој учесници узастопно играју  $n$  турнира те је потребно да записујемо побједника у виду низа кодова, а циљ је да оптимизујемо дужину укупне поруке. У првом моделу, она ће бити дужине  $2n$ , а како  $n$  постаје велико, други модел у просјеку даје код дужине  $\frac{7}{4}n$ .

## 1.2 Ентропија

Шта представља доњу границу за компресију података? Интуитивно нам је јасно да не можемо у недоглед вршити компресије, те да, када елиминишемо неку површинску редундантност, стижемо до сржи, те даље смањење није могуће. У наставку ћемо пробати да квантификујемо количину информације коју подаци носе.

Замислимо Бернулијев експеримент - бацање новчића са два исхода, при чему је шанса да добијемо главу  $p$ , а писмо  $1 - p$ . У случају да је  $p = \frac{1}{2}$ , исход је потпуно насумичан, те је јасно да даљу компресију не можемо извршити. Ако је, ипак  $p = \frac{1}{10}$ , исход „глава” носи значајну информацију, јер је риједак, а исход „писмо” нема неку тежину јер је то нешто што је врло честа појава. Дакле, што је исход мање вјероватан, он носи већу информацију за нас, а они врло вјероватни исходи су практично безначајни.<sup>2</sup>

---

<sup>1</sup>Кодови морају бити *префиксни*, тј. ниједна кодна ријеч не смије бити почетак неке друге. Само тако се низ спојених кодних ријечи може једнозначно декодирати.

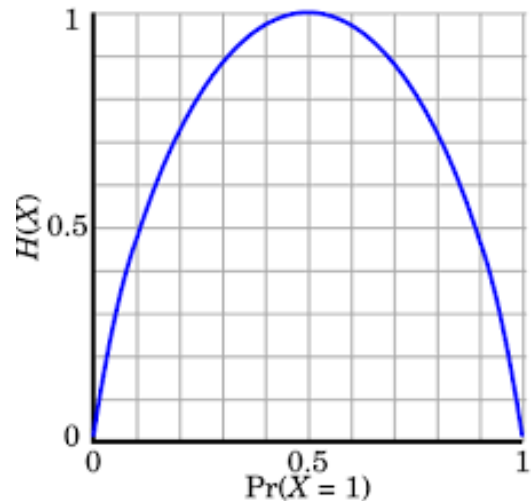
<sup>2</sup>Упоредимо информације које носе следеће двије реченице:

Дакле, за одређен исход, функција која одређује информацију коју носи тај исход догађаја<sup>3</sup> мора бити опадајућа у односу на вјероватноћу тог догађаја.

Функција  $i(x) = -\log_2 p(x)$ <sup>4</sup> је једна функција која испуњава претходни услов. Поред тога, она испуњава и сљедећи битан услов: ако имамо два *независна* догађаја  $x$  и  $y$ , интуитивно је да је информација коју носи одигравање оба та догађаја једнака збиру информација које носе ти догађаји појединачно. Функција то одражава јер, због независности ( $p(xy) = p(x)p(y)$ ), вриједи:

$$\begin{aligned} i(xy) &= -\log_2 p(xy) \\ &= -\log_2 p(x)p(y) \\ &= -\log_2 p(x) - \log_2 p(y) \\ &= i(x) + i(y) \end{aligned}$$

За неку дискретну случајну величину  $X$ , дефинишимо њену **ен-**



Функција ентропије  
 $H(x) = -x \log_2 x$

---

*Данас је у Сахари падала киша.*

*Данас је у Енглеској падала киша.*

Очигледно је друга вјероватнија, а самим тим носи мање информације

<sup>3</sup>Ту функцију, *информацију догађаја*  $x$ , означавамо са  $i(x)$ .

<sup>4</sup>Може се, уз неке додатне претпоставке, извести да је, до на избор основе логаритма, ово једина таква функција. Више прочитајте на линку

*ентропију* као усредњену информацију сваког могућег исхода,<sup>5</sup> тј:

$$H(X) = - \sum_{i=1}^n p_i \cdot \log_2 p_i$$

Сада нам је јасно да је она неко мјерило простора који је потребан да би се пренијела информација о исходу догађаја  $X$  - испоставиће се да је она баш најмања просјечна дужина бинарног кода по симболу којим се то постиже без губитка података, кад кодирамо дугачке низове независних исхода.

### 1.2.1 Примјер

1. За случај бацања поштеног новчића, ентропију можемо пронаћи као

$$H(X_1) = -\frac{1}{2} \log_2 \frac{1}{2} - \frac{1}{2} \log_2 \frac{1}{2} = 1$$

Ово се слаже са нашим досадашњим разматрањем - потребан је тачно 1 бит информације за то.

2. С друге стране, када новчић није поштен, са вјероватноћом  $p = \frac{1}{10}$ , добили бисмо

$$H(X_2) = -\frac{1}{10} \log_2 \frac{1}{10} - \frac{9}{10} \log_2 \frac{9}{10} \approx 0.47,$$

што такође има смисла, јер је овдје један догађај врло вјероватан, и носи мање информације, па у просјеку и за сам експеримент је мања неодређеност. У екстремуму, кад би новчић увијек падао на главу, тј.  $p = 1$ , тада би израз за ентропију (у лимесу) тежио 0 - што је у реду, јер о њему није ни потребно преносити информацију - исход је унапријед познат.

---

<sup>5</sup>Користимо конвенцију  $0 \cdot \log_2 0 = 0$ . Она је природна јер је  $\lim_{x \rightarrow 0^+} x \log_2 x = 0$ , па је функција  $x \mapsto -x \log_2 x$  непрекидна на  $[0, 1]$  ако јој у нули додијелимо вриједност 0. Другим ријечима, исходи вјероватноће 0 не доприносе ентропији.

## 1.3 Шенонова теорема о кодирању извору

Сада можемо да одговоримо на питање из претходног дијела - гдје је доња граница компресије? Прво прецизирајмо шта под кодом подразумевамо.

**Дефиниција 1.3.1.** Код низа  $X^n$  је пресликавање које сваком низу  $\mathbf{x} \in \mathcal{X}^n$  додјељује бинарну ријеч дужине  $L(\mathbf{x})$ . Код је **префиксан** ако ниједна кодна ријеч није почетак неке друге.

Одговор нам даје сљедећа теорема:

**Теорема 1.3.2** (*Шенонова теорема о кодирању извора*).

Нека је дато  $\epsilon > 0$  и нека су  $X_1, \dots, X_n$  независне случајне величине исте расподеле  $X$  (*iid*). Тада за сваки префиксан код низа  $X^n = X_1 X_2 \dots X_n$  вриједи да је очекивана дужина кода бар  $nH(X)$ , али за довољно велико  $n$  постоји код којим се тој граници можемо произвољно приближити, тј:

$$H(X) \leq \frac{E[L(X^n)]}{n} \leq H(X) + \epsilon$$

Означимо са  $\mathcal{X}$  (коначан) скуп могућих вриједности величине  $X$ . Посматраћемо неких  $n$  случајних величина  $X_1, \dots, X_n$  (*iid*), и понашање низа када  $n$  тежи бесконачности. Важи:

**Лема 1.3.3** (*Асимптотска еквипартиција вјероватноће*).

Ако су  $X_1, \dots, X_n$  (*iid*) случајне величине, тада

$$-\frac{1}{n} \log_2 p(X_1, \dots, X_n) \rightarrow H(X), \quad n \rightarrow \infty,$$

по вјероватноћи, тј. за свако  $\epsilon > 0$  је

$$P \left\{ \left| -\frac{1}{n} \log_2 p(X_1, \dots, X_n) - H(X) \right| < \epsilon \right\} \rightarrow 1.$$

*Доказ.* Случајне величине су независне, па важи  $p(x_1, \dots, x_n) = \prod_{i=1}^n p(x_i)$ . Величине  $\log_2 p(X_i)$  су независне, исто расподијељене и, јер је абецеда  $\mathcal{X}$  коначна, имају коначно очекивање, па се по слабом закону великих бројева њихова аритметичка средина приближава очекивању (по вјероватноћи):

$$\begin{aligned}
-\frac{1}{n} \log_2 p(X_1, \dots, X_n) &= -\frac{1}{n} \log_2 \left( \prod_{i=1}^n p(X_i) \right) \\
&= -\frac{1}{n} \sum_{i=1}^n \log_2 p(X_i) \\
&\xrightarrow{n \rightarrow \infty} -E[\log_2 p(X)] \quad (\text{по слабом закону великих бројева}) \\
&= -\sum_{x \in \mathcal{X}} p(x) \log_2 p(x) \\
&= H(X)
\end{aligned}$$

□

**Дефиниција 1.3.4.** За дато  $\epsilon > 0$ , **типичан скуп**  $A_\epsilon^{(n)}$  је скуп низова  $(x_1, \dots, x_n) \in \mathcal{X}^n$  таквих да:

$$\frac{1}{2^{n(H(X)+\epsilon)}} \leq p(x_1, \dots, x_n) \leq \frac{1}{2^{n(H(X)-\epsilon)}}$$

**Лема 1.3.5.** Типичан скуп  $A_\epsilon^{(n)}$  има следећа својства

1.  $P\{A_\epsilon^{(n)}\} > 1 - \epsilon$ , за довољно велико  $n$
2.  $|A_\epsilon^{(n)}| \leq 2^{n(H(X)+\epsilon)}$
3.  $(1 - \epsilon)2^{n(H(X)-\epsilon)} \leq |A_\epsilon^{(n)}|$ , за довољно велико  $n$

*Доказ.*

1. По леми 1.3.3, за свако  $\delta > 0$  постоји  $n_0$  тако да за свако  $n > n_0$  вриједи:

$$P \left\{ \left| -\frac{1}{n} \log_2 p(X_1, \dots, X_n) - H(X) \right| < \epsilon \right\} > 1 - \delta$$

Унутрашњи услов је еквивалентан са  $2^{-n(H(X)+\epsilon)} < p(X_1, \dots, X_n) < 2^{-n(H(X)-\epsilon)}$ , па је догађај у загради садржан у догађају  $\{(X_1, \dots, X_n) \in A_\epsilon^{(n)}\}$  (строге неједнакости су посебан случај нестрогих из дефиниције). Зато је  $P\{A_\epsilon^{(n)}\} > 1 - \delta$ , а постављајући  $\delta = \epsilon$ , добијамо тражени услов.

2.

$$\begin{aligned} 1 &= \sum_{\mathbf{x} \in \mathcal{X}^n} p(\mathbf{x}) \geq \sum_{\mathbf{x} \in A_\epsilon^{(n)}} p(\mathbf{x}) \\ &\geq \sum_{\mathbf{x} \in A_\epsilon^{(n)}} 2^{-n(H(X)+\epsilon)} \\ &= |A_\epsilon^{(n)}| \cdot 2^{-n(H(X)+\epsilon)} \end{aligned}$$

3. Користећи дио 1. добијамо да за довољно велико  $n$ :

$$\begin{aligned} 1 - \epsilon &\leq P\{A_\epsilon^{(n)}\} \leq \sum_{\mathbf{x} \in A_\epsilon^{(n)}} 2^{-n(H(X)-\epsilon)} \\ &= |A_\epsilon^{(n)}| \cdot 2^{-n(H(X)-\epsilon)} \end{aligned}$$

□

Посљедица ових двију лема је управо Шенонова теорема (теорема 1.3.2), тачније њена горња граница.

*Доказ.* Нека је  $\epsilon > 0$  и нека је  $\delta > 0$ , које ћемо на крају избрати довољно малим. Подијелимо скуп  $\mathcal{X}^n$  на типичан скуп  $A_\delta^{(n)}$  и

његов комплемент. Цијели низ  $X^n$  кодирамо као једну ријеч, па је  $E[L(X^n)]/n$  просјечна дужина кода по симболу.

По леми 1.3.5 у типичном скупу има највише  $2^{n(H(X)+\delta)}$  елемената, па је за редни број елемента довољно  $\lceil n(H(X)+\delta) \rceil \leq n(H(X)+\delta)+1$  цифара. Зато њих кодирамо са највише  $n(H(X) + \delta) + 2$  цифара (прва цифра је 1, а остатак је редни број елемента у типичном скупу - на било који начин, нпр. лексикографски). Комплемент кодирамо са највише  $n \log_2 |\mathcal{X}| + 2$  цифара (као малоприје - започнемо код цифром 0, а остатак је редни број низа у  $\mathcal{X}^n$ , за шта је довољно  $\lceil n \log_2 |\mathcal{X}| \rceil \leq n \log_2 |\mathcal{X}| + 1$  цифара). Одредимо очекивање дужине кода.

$$\begin{aligned}
E[L(X^n)] &= \sum_{\mathbf{x} \in A_\delta^{(n)}} p(\mathbf{x})[n(H(X) + \delta) + 2] + \sum_{\mathbf{x} \notin A_\delta^{(n)}} p(\mathbf{x})[n \log_2 |\mathcal{X}| + 2] \\
&= P\{A_\delta^{(n)}\}[n(H(X) + \delta) + 2] + P\{(A_\delta^{(n)})^c\}[n \log_2 |\mathcal{X}| + 2] \\
&\leq [n(H(X) + \delta) + 2] + \delta[n \log_2 |\mathcal{X}| + 2] \\
&= n \left[ H(X) + \delta + \delta \log_2 |\mathcal{X}| + \frac{2}{n} + \frac{2\delta}{n} \right] \\
&= n[H(X) + \delta']
\end{aligned}$$

Овдје смо у трећем реду користили да је  $P\{A_\delta^{(n)}\} \leq 1$  и  $P\{(A_\delta^{(n)})^c\} < \delta$  (по леми 1.3.5), а  $\delta' = \delta + \delta \log_2 |\mathcal{X}| + \frac{2+2\delta}{n}$ . Изаберимо  $\delta$  тако да је  $\delta + \delta \log_2 |\mathcal{X}| < \frac{\epsilon}{2}$ , а затим  $n$  довољно велико да је  $\frac{2+2\delta}{n} < \frac{\epsilon}{2}$ . Тада је  $\delta' < \epsilon$ , па за довољно велико  $n$  постоји код за који је  $E[L(X^n)] \leq nH(X) + n\epsilon$ , што је горња граница из теореме. Добијени код је префиксан: прва цифра одређује којој групи ријеч припада, а унутар групе редне бројеве записујемо са једнаким бројем цифара.  $\square$

Преостаје доња граница, до које долазимо помоћу Крафтове неједнакости.

**Лема 1.3.6** (*Крафтова неједнакост*).

Ако су  $l_1, \dots, l_m$  дужине кодних ријечи неког префиксног кода, тада је  $\sum_{i=1}^m 2^{-l_i} \leq 1$ .

*Доказ.* Код представимо бинарним стаблом: сваку кодну ријеч читамо као пут од коријена, при чему цифра 0 значи прелазак у лијево, а цифра 1 у десно дијете, а чвор у коме пут завршава одговара тој ријечи, па је његова дубина једнака дужини ријечи. Пошто је код префиксан, ниједна кодна ријеч није почетак друге, тј. ниједан од ових чворова није предак другог, па су сви они листови. Нека је  $l_{\max}$  највећа дужина. Кодна ријеч дужине  $l_i$  има  $2^{l_{\max}-l_i}$  потомака на нивоу  $l_{\max}$ , и ти скупови потомака су дисјунктни за различите кодне ријечи. Како на том нивоу укупно има  $2^{l_{\max}}$  чворова, важи  $\sum_i 2^{l_{\max}-l_i} \leq 2^{l_{\max}}$ , а дијелењем са  $2^{l_{\max}}$  добијамо тврдњу.  $\square$

*Доказ.* Доња граница теореме. Нека су  $p_i$  вјероватноће низова из  $\mathcal{X}^n$  (само оних за које је  $p_i > 0$ ), а  $l_i$  дужине њихових кодних ријечи. Функција  $\log_2$  је конкавна, па по Јенсеновој неједнакости вриједи:

$$\begin{aligned} E[L(X^n)] - H(X^n) &= \sum_i p_i l_i + \sum_i p_i \log_2 p_i = - \sum_i p_i \log_2 \frac{2^{-l_i}}{p_i} \\ &\geq - \log_2 \sum_i p_i \cdot \frac{2^{-l_i}}{p_i} = - \log_2 \sum_i 2^{-l_i} \geq 0, \end{aligned}$$

гдје посљедња неједнакост слиједи из леме 1.3.6. Због независности је  $\log_2 p(X_1, \dots, X_n) = \sum_{j=1}^n \log_2 p(X_j)$ , па је  $H(X^n) = E[-\log_2 p(X^n)] = \sum_{j=1}^n E[-\log_2 p(X_j)] = nH(X)$ , одакле слиједи  $E[L(X^n)] \geq nH(X)$ .  $\square$

Шта нам говори Шенонова теорема? Теоријски минимум за кодирање неке случајне величине  $X$  (у просјеку) је  $H(X)$ . Дакле, не можемо боље од тога, а да не губимо информацију. Ово је значајно за компресију без губљења података (*тзв. lossless compression*), што је

на примјер реалан захтјев код компресије текста, док код компресије слика и видеа често није потребно да очувамо 1:1 изворни квалитет.

Али да ли ово значи да за неки конкретан захтјев (нпр. компресију текста, гдје унапријед знамо фреквенцију појављивања слова у српском језику), не можемо боље од дате границе? Одговор је: Не! Сјетимо се нашег услова,  $X$  (*iid*) - независне случајне величине исте расподеле. Међутим, у нашем језику тако нешто не вриједи - слова се не појављују независно једно од других, ако у неком тексту имамо слово 'ј', највећа је шанса да иза њега слиједи слово 'е', док је врло мала шанса да је иза њега слово 'ж' (али не и 0, нпр. *нај~~ж~~есићи*). Слично овоме, у компресији текста можемо користити и одређеност у томе која ријеч слиједи тренутну - нпр. ријеч *нова* ће вјероватно бити праћена ријечју *година*, а никада ријечју *острвима*.

Дакле,  $H(X)$  је теоријски минимум само под претпоставком да су симболи независни и једнако расподијељени. Кад та претпоставка не важи, границу можемо пробити тако што искористимо зависност међу симболима. Најједноставнији примјер је низ *abababab . . .*: слова *a* и *b* се појављују подједнако често, па је  $H(X) = 1$  бит по слову, али је свако слово одређено претходним. Умјесто да сваки знак записујемо посебно, довољно је записати *ab*, *понови n пута*. Мјера те зависности је *условна ентропија*  $H(X_2|X_1)$ , која је за овај низ једнака 0.

Овај приступ користе *рјечничке методе* - дијелове текста који су се већ појавили замјењују упућивањем на ранију појаву, а расподелу симбола не морају да знају. У поглављу *Експериментални резултати* ћемо видјети и колико то побољшање доноси у пракси.

# Хафманово кодирање

## 2.1 Мотивација

Шенонова теорема каже колика је граница компресије, али не и како доћи до кода који јој се приближава. Зато се поставља питање: који је најбољи префиксан код за дату расподелу? Одговор даје Хафманов алгоритам.

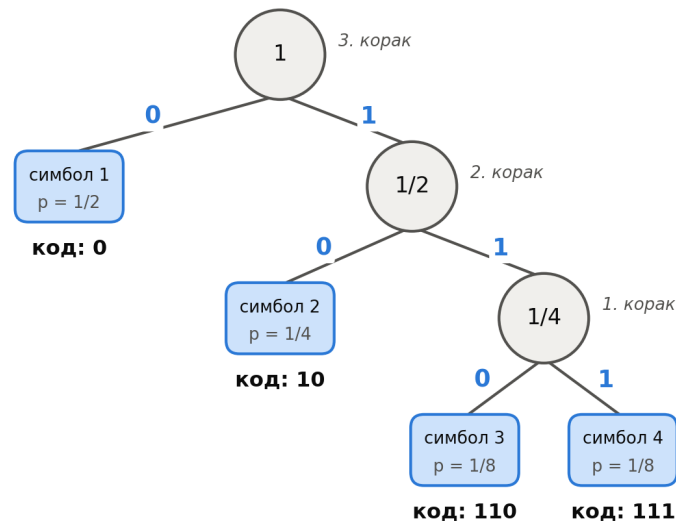
## 2.2 Алгоритам

Нека су дати симболи  $1, \dots, m$  са вјероватноћама  $p_1, \dots, p_m$ . Хафманов алгоритам гради бинарно стабло од листова ка корену:

1. Сваки симбол је стабло са једним чвором, тежине  $p_i$ .
2. Све док има више од једног стабла, узмемо два стабла најмање тежине и спојимо их под нови чвор, чија је тежина њихов збир.
3. Кодна ријеч симбола је пут од коријена до његовог листа (лијево дијете је 0, десно 1).

За расподелу из увода прво спајамо симболе 3 и 4 (тежина  $\frac{1}{4}$ ), затим добијено стабло и симбол 2 (тежина  $\frac{1}{2}$ ), и на крају добијено

стабло и симбол 1. Добијамо управо код 0, 10, 110, 111 из увода, за који је  $E[L] = \frac{7}{4} = H(X)$ . Ако се стабла чувају у приоритетном реду (*min-heap*), сваки корак кошта  $O(\log m)$ , па је укупна сложеност  $O(m \log m)$ .



Хафманово стабло за расподелу из увода. Број у чвору је његова тежина, а цифре на гранама дају кодне ријечи (лијево 0, десно 1). Чворови настају редом од најмање тежине: прво се спајају симболи 3 и 4.

Описани алгоритам је *статички* Хафманов код<sup>1</sup>: стабло се гради тек кад су познате фреквенције свих симбола, па се текст мора прочитати два пута, а фреквенције (или само стабло) морају бити записане уз компримовани текст. Постоји и *динамички* (адаптивни) Хафманов код (алгоритми FGK и Vitter), који то избјегава. Кодер и декодер почињу са стаблом које садржи само посебан чвор за симболе који још нису послати. Кад се симбол први пут појави, шаље се код тог посебног чвора, па сâм симбол, а иначе се шаље његов код

<sup>1</sup>Анимирана визуелизација (бројање, изградња стабла, кодирање и декодирање): [chrislaux.com/huffman](http://chrislaux.com/huffman).

у тренутном стаблу. Затим се тежина листа тог симбола повећа за један и стабло се измијени (замјеном чворова) тако да поново буде Хафманово. Кодер и декодер примјењују иста правила, па су стабла стално усаглашена, тако да је довољан један пролаз кроз текст и не шаље се таблица фреквенција. Ову варијанту у раду нећемо обрађивати.

## 2.3 Оптималност

**Лема 2.3.1.** *Нека су  $x$  и  $y$  два симбола најмање вјероватноће. Тада постоји оптималан префиксан код у којем су кодне ријечи ових симбола најдуже и разликују се само у посљедњој цифри.*

*Доказ.* Узмемо оптималан код и представимо га бинарним стаблом (као у доказу Крафтове неједнакости). У њему сваки унутрашњи чвор има оба дјетета (у супротном бисмо чвор избацили и скратили кодне ријечи, чиме би  $E[L]$  опало). Нека је  $a$  лист највеће дубине; његов сестрински чвор  $b$  је такође лист исте дубине. Замијенимо кодне ријечи симбола  $x$  и  $a$ . Промјена очекиване дужине је  $(p_x - p_a)(l_a - l_x) \leq 0$ , јер је  $p_x \leq p_a$  и  $l_a \geq l_x$ , па је код и даље оптималан. Аналогно замијенимо  $y$  и  $b$ . Сада су  $x$  и  $y$  сестрински чворови на највећој дубини.  $\square$

**Теорема 2.3.2.** *Хафманов код је оптималан, тј. има најмању очекивану дужину међу свим префиксним кодовима за дату расподелу.*

*Доказ.* Индукцијом по броју симбола  $m$ . За  $m = 2$  једини (па и оптималан) код је  $0, 1$ . Нека је  $m > 2$ , нека су  $x, y$  два симбола најмање вјероватноће и нека је  $z$  нови симбол вјероватноће  $p_z = p_x + p_y$  који их замјењује. Први корак Хафмановог алгоритма спаја

$x$  и  $y$ , а даље ради као алгоритам за  $m - 1$  симбола, па је Хафманов код  $C$  добијен од Хафмановог кода  $C'$  за симболе са  $z$  тако што се кодна ријеч симбола  $z$  продужи цифром 0 (за  $x$ ) и 1 (за  $y$ ). Тада су  $x$  и  $y$  за један ниво дубље од  $z$ , па је

$$E[L_C] = E[L_{C'}] + p_x + p_y.$$

Нека је  $D$  оптималан код за полазне симболе у којем су  $x, y$  браћа (постоји по леми 2.3.1) и нека је  $D'$  код добијен спајањем  $x$  и  $y$  у  $z$ . Исто као малоприје  $E[L_D] = E[L_{D'}] + p_x + p_y$ . По индуктивној претпоставци  $C'$  је оптималан, па је  $E[L_{D'}] \geq E[L_{C'}]$ , одакле

$$E[L_D] \geq E[L_{C'}] + p_x + p_y = E[L_C].$$

Како је  $D$  оптималан, то је и  $C$  оптималан.  $\square$

## 2.4 Границе

**Теорема 2.4.1.** *За Хафманов код  $C$  расподеле  $X$  вриједи  $H(X) \leq E[L_C] < H(X) + 1$ .*

*Доказ.* Доња граница је доказана у доказу Шенонове теореме (за  $n = 1$ ). За горњу, узмемо дужине  $l_i = \lceil -\log_2 p_i \rceil$ . Тада је  $\sum_i 2^{-l_i} \leq \sum_i p_i = 1$ , па постоји префиксан код са тим дужинама.<sup>2</sup> За њега је  $E[L] = \sum_i p_i l_i < \sum_i p_i (-\log_2 p_i + 1) = H(X) + 1$ . Хафманов код је оптималан, па је његова очекивана дужина највише толика.  $\square$

Кодирањем блокова  $X^n$  ово даје просјечну дужину по симболу мању од  $H(X) + \frac{1}{n}$ , што показује цијену кодирања симбол по симбол: до једне цифре изнад ентропије.

---

<sup>2</sup>Ово је обрат Крафтове неједнакости, видјети нпр. Cover & Thomas, *Elements of Information Theory*, поглавље 5.

## 2.5 Имплементација

Алгоритам смо имплементирали у језику C++ (као и све остале алгоритме у раду), при чему су вјероватноће замијењене фреквенцијама бајтова у тексту. Ријеч је о *статичкој* варијанти (текст се чита два пута), а фреквенције се записују на почетак датотеке, па декодер гради исто стабло као кодер. Нека је  $n$  дужина текста, а  $k \leq 256$  број различитих бајтова. Кодирање је  $O(n + k \log k)$ , а декодирање  $O(k \log k + B)$ , гдје је  $B$  дужина кодираног дијела у битовима. Пошто је просјечна дужина кода мања од  $H(X) + 1 \leq 9$  бита, важи  $B \leq 9n$ , па су оба  $O(n)$ . Додатна меморија је  $O(k^2)$ , што је за  $k \leq 256$  константна сложеност. Кодирање и декодирање у псеудокоду:

```
1: function BUILD_TREE(freq)
2:    $Q \leftarrow$  min-heap of leaves ( $s, freq[s]$ ), for all  $s$  with  $freq[s] > 0$ 
3:   while  $|Q| > 1$  do
4:      $a, b \leftarrow$  remove the two nodes of smallest weight from  $Q$ 
5:     insert into  $Q$  a new node with children  $a, b$  and weight  $a.w + b.w$ 
6:   end while
7:   return the only node in  $Q$ 
8: end function

9: function HUFFMAN_ENCODE(text)
10:   $freq \leftarrow$  number of occurrences of each byte in text
11:   $root \leftarrow$  BUILD_TREE(freq)
12:  write the header (length of text and freq)
13:  for each byte  $c$  of text do
14:    write the path from  $root$  to leaf  $c$  (left = 0, right = 1)
15:  end for
16: end function

17: function HUFFMAN_DECODE(data)
18:  read  $n$  and freq from the header;  $root \leftarrow$  BUILD_TREE(freq)
19:  for  $i \leftarrow 1$  to  $n$  do
20:     $v \leftarrow root$ 
21:    while  $v$  is not a leaf do
```

```

22:          $v \leftarrow$  left or right child of  $v$ , according to the next bit of  $data$ 
23:     end while
24:     output the symbol of leaf  $v$ 
25: end for
26: end function

```

Једнаке тежине у приоритетном реду разрјешавају се по редном броју чвора, па кодер и декодер граде исто стабло. Случај само једног различитог бајта обрађујемо посебно (излаз је само заглавље).

У пракси се Хафманов код ријетко користи сам; најчешће је завршни степен других формата: DEFLATE (gzip, ZIP, PNG), *bzip2*, JPEG и MP3.

# Рјечничке методе

## 3.1 Мотивација

Хафманов код користи само фреквенције појединачних симбола, а не гледа шта се око њих налази. Текст је, међутим, пун понављања: исте ријечи и дијелови ријечи појављују се изнова, а то смо већ наслутили на крају првог поглавља. *Рјечничке методе*, које су увели Зив и Лемпел (Ziv, Lempel), понављање користе директно: дио текста који се већ појавио замјењују упућивањем на ранију појаву. Не подразумијевају познавање расподеле симбола унапријед. Разматрамо три методе: LZ77, LZ78 и LZW.<sup>1</sup>

## 3.2 LZ77

Метода памти посљедњих  $W$  прочитаних знакова (*клизни прозор*). У сваком кораку, за текст  $x_i x_{i+1} \dots$  који тек треба кодирати, налази се најдужи префикс који почиње негдје у прозору. Ако је то поклапање дужине  $l$  на удаљености  $d$  уназад, а  $c$  знак који слиједи, у излаз се пише тројка  $(d, l, c)$ , а кодирање се наставља од позиције

---

<sup>1</sup>Анимирана визуелизација све три методе (кодирање и декодирање корак по корак): [chrislaux.com/lempelziv](http://chrislaux.com/lempelziv).

$i + l + 1$ . Ако поклапања нема, пише се  $(0, 0, c)$ . За текст *abracadabra* добијамо

$(0, 0, a), (0, 0, b), (0, 0, r), (3, 1, c), (5, 1, d), (7, 3, a)$ .

*Декодирање* је једноставно: за тројку  $(d, l, c)$  копирамо  $l$  знакова почевши од  $d$  мјеста уназад, па додамо  $c$ . Копирамо знак по знак, јер се поклапање може преклапати са оним што се управо пише ( $l > d$ ): за *aaaa* добијамо  $(0, 0, a), (1, 2, a)$ . Исправност слиједи индукцијом: након сваке тројке декодирани низ је префикс полазног текста, јер кодер поклапање тражи управо у већ кодираном дијелу, па декодер копира исте знакове.

```

1: function LZ77_ENCODE(text)
2:    $i \leftarrow 1$ 
3:   while  $i \leq |text|$  do
4:      $(d, l) \leftarrow \text{LONGEST\_MATCH}(text, i)$ 
5:     write  $(d, l, text[i + l])$ 
6:      $i \leftarrow i + l + 1$ 
7:   end while
8: end function

9: function LZ77_DECODE(data)
10:  text  $\leftarrow$  empty string
11:  for each triple  $(d, l, c)$  in data do
12:    for  $k \leftarrow 1$  to  $l$  do
13:      append  $text[|text| - d + 1]$  to text
14:    end for
15:    append  $c$  to text
16:  end for
17: end function

```

Функција `longest_match` враћа удаљеност  $d$  и дужину  $l$  најдужег префикса текста од позиције  $i$  који почиње у прозору (у  $l$  се не рачуна посљедњи знак текста), односно  $(0, 0)$  ако таквог нема.

*Претрага поклапања* је једини захтјеван корак. Нека је  $L$  највећа дужина поклапања. Наивна претрага за сваку од  $W$  почетних

позиција пореди знак по знак, што је  $O(WL)$  по кораку. Помоћу КМР алгоритма (префиксна функција низа  $x_i \dots x_{i+L-1}$  и један пролаз кроз прозор) корак је  $O(W + L)$ . Корака има највише  $n$ , па је са КМР кодирање  $O(n(W + L))$ , тј.  $O(nW)$  јер је  $L \leq W$ . Исто вриједи и за наивну претрагу: један њен корак кошта највише  $W$  пута број знакова које тај корак потроши (ниједан кандидат нема дуже поклапање од изабраног), а збир потрошених знакова по свим корацима је  $n$ , па је и она  $O(nW)$ . Дакле, у најгорем случају су обје претраге истог реда величине. Разлика је на подацима са дугим поклапањима: тада корака има мање (око  $n/L$ ), а КМР по кораку кошта  $O(W + L)$ , па је до око  $L$  пута бржи. Декодирање је  $O(n)$ , јер се сваки излазни знак пише тачно једном. Обје претраге дају исти излаз; у пракси је наивна често бржа, јер поређење већином не успије већ на првом знаку, док КМР добија на јако понављајућим подацима (види поглавље о експериментима).

LZ77 је основа формата DEFLATE (gzip, ZIP, PNG), а на њему су засновани и LZ4, Snappy, LZMA (xz, 7-Zip) и zstd.

### 3.3 LZ78

Умјесто прозора, LZ78 гради *рјечник фразе*, који је на почетку празан. У сваком кораку налази се најдужа фраза из рјечника која је префикс преосталог текста, пише се пар  $(i, c)$  ( $i$  је индекс те фразе, 0 за празну, а  $c$  наредни знак), а фраза допуњена знаком  $c$  додаје се у рјечник као нова. За *abracadabra* фразе су **a**, **b**, **r**, **ac**, **ad**, **ab**, **ra**, па је излаз

$(0, \mathbf{a}), (0, \mathbf{b}), (0, \mathbf{r}), (1, \mathbf{c}), (1, \mathbf{d}), (1, \mathbf{b}), (3, \mathbf{a})$ .

Рјечник се чува као *префиксно дрво* (*trie*): чвор је фраза, а дијете чвора добија се додавањем једног знака. Налажење најдуже фразе је спуштање низ дрво, а свака нова фраза је један нови чвор. Зато је

очекувана сложеност кодирања  $O(n)$  (уз хеш табелу за дјецу чвора). Декодирање је такође  $O(n)$ : за исписивање једне фразе потребно је онолико корака колика је њена дужина (пењемо се до корена), а збир дужина свих фраза је  $n$ .

*Декодер* гради исти рјечник: сваки пар  $(i, c)$  даје фразу која се састоји од фразе са индексом  $i$  и знака  $c$ , коју исписује и додаје у рјечник. Кодер и декодер зато у сваком кораку имају исти рјечник, одакле слиједи исправност. Ако се текст заврши усред фразе која већ постоји у рјечнику, пише се њен индекс са произвољним знаком, а декодер вишак одбацује јер зна дужину текста.

```

1: function LZ78_ENCODE(text)
2:   dict  $\leftarrow$  dictionary containing only the empty phrase  $\varepsilon$ , with index 0
3:   w  $\leftarrow \varepsilon$  ▷ the current phrase
4:   for each character c of text do
5:     if wc is in dict then
6:       w  $\leftarrow wc$ 
7:     else
8:       write (dict[w], c)
9:       add wc to dict with the next free index
10:      w  $\leftarrow \varepsilon$ 
11:    end if
12:  end for
13:  if w  $\neq \varepsilon$  then
14:    write (dict[w], any character)
15:  end if
16: end function

17: function LZ78_DECODE(data)
18:   dict  $\leftarrow$  dictionary containing only the empty phrase  $\varepsilon$ , with index 0
19:   text  $\leftarrow$  empty string
20:   for each pair  $(i, c)$  in data do
21:     w  $\leftarrow$  dict[i] followed by c
22:     append w to text
23:     add w to dict with the next free index
24:   end for
25: end function

```

LZ78 се у чистом облику ријетко користи; важнији је као основа за LZW и као теоријски модел.

## 3.4 LZW

LZW је измјена LZ78 коју је предложио Велч (Welch). Рјечник на почетку садржи свих 256 појединачних бајтова, па у излаз иду *само индекси фразе*, без додатног знака. Нова фраза (претходна фраза и први знак наредне) додаје се у рјечник, а тај знак постаје почетак наредне фразе. Тако се избјегава писање знака уз сваку фразу, што је разлог зашто је LZW у нашим мјерењима бољи од LZ78. За *abracadabra* излаз је

a, b, r, a, c, a, d, 256, 258,

гдје су 256 и 258 фразе *ab* и *ra* додате раније. Имплементација користи исту структуру као LZ78. Зато је и кодирање сложености  $O(n)$ , а декодирање  $O(n)$ , јер се свака фраза исписује пењањем до корена, а збир дужина фразе је  $n$ .

```
1: function LZW_ENCODE(text)
2:   dict  $\leftarrow$  dictionary of all single bytes (indices 0, ..., 255)
3:   w  $\leftarrow$  first character of text
4:   for each next character c of text do
5:     if wc is in dict then
6:       w  $\leftarrow$  wc
7:     else
8:       write dict[w]
9:       add wc to dict with the next free index
10:      w  $\leftarrow$  c
11:     end if
12:   end for
13:   write dict[w]
14: end function
```

```

15: function LZW_DECODE(data)
16:   dict  $\leftarrow$  dictionary of all single bytes (indices 0, ..., 255)
17:   prev  $\leftarrow$  first code of data; output dict[prev]
18:   for each next code k of data do
19:     if k is in dict then
20:       w  $\leftarrow$  dict[k]
21:     else  $\triangleright$  k is the entry the encoder has just added
22:       w  $\leftarrow$  dict[prev] followed by its first character
23:     end if
24:     output w
25:     add dict[prev] followed by the first character of w to dict
26:     prev  $\leftarrow$  k
27:   end for
28: end function

```

LZW се користи у формату GIF, у формату TIFF (као један од начина компресије), у PDF-у и у Unix алату **compress** (датотеке **.Z**).

## 3.5 Поређење

Све три методе ослањају се на понављање, али га чувају различито.

|            | LZ77           | LZ78           | LZW            |
|------------|----------------|----------------|----------------|
| излаз      | $(d, l, c)$    | $(i, c)$       | $i$            |
| меморија   | прозор ( $W$ ) | рјечник, расте | рјечник, расте |
| вријеме    | $O(n(W + L))$  | $O(n)$         | $O(n)$         |
| кодирања   |                |                |                |
| вријеме    | $O(n)$         | $O(n)$         | $O(n)$         |
| декодирања |                |                |                |

LZ77 се прилагођава промјенама у тексту, јер заборавља оно што је ван прозора, а LZ78 и LZW памте фразе и брзо кодирају, али им и рјечник расте. Важно је напоменути да LZ77 и Хафманов могу да се комбинују: излаз LZ77 (слова, дужине и удаљености) није равномерно распоређен, па се накнадно кодира Хафмановим кодом. Та

комбинација је позната као *DEFLATE*, формат који користе *gzip*, *ZIP* и *PNG*.

# Експериментални резултати

## 4.1 Поставка

Поредили смо Хафманов код, LZ77 (са КМР и наивном претрагом), LZ78 и LZW на улазима различите природе: текст на ћирици и на енглеском (чланци са Википедије су изворни текст са вики ознакама), изворни код, енглески чланак поновљен осам пута, периодичан текст, милион пута слово а и милион случајних бајтова. За сваки улаз програм провјерава и да декодирањем добијамо полазни садржај (у свим случајевима јесмо). Величине укључују заглавља, а обрачун је у битовима по бајту.

## 4.2 Степен компресије

| улаз                             | величина (B) | $H(X)$ | $H(X_2 X_1)$ | Хафман | LZ77  | LZ78 | LZW  |
|----------------------------------|--------------|--------|--------------|--------|-------|------|------|
| почетак овог рада (ћир.)         | 18 052       | 4.96   | 2.59         | 5.26   | 4.06  | 4.40 | 3.51 |
| Београд, вики (ћир.)             | 156 135      | 5.25   | 2.93         | 5.31   | 3.90  | 3.88 | 3.11 |
| <i>Information theory</i> , вики | 88 580       | 5.29   | 3.84         | 5.38   | 4.94  | 4.67 | 3.82 |
| изворни код C++                  | 1 000 000    | 4.95   | 3.54         | 4.98   | 2.51  | 3.12 | 2.40 |
| енглески чланак $\times 8$       | 708 640      | 5.29   | 3.84         | 5.33   | 4.91  | 3.53 | 2.75 |
| периодичан текст                 | 1 003 322    | 3.36   | 0.02         | 3.51   | 0.11  | 0.13 | 0.08 |
| слово <b>a</b> $\times 10^6$     | 1 000 000    | 0      | 0            | 0.00   | 0.11  | 0.03 | 0.02 |
| случајни бајтови                 | 1 000 000    | 8.00   | 7.95         | 8.01   | 13.59 | 9.04 | 9.55 |

Вриједности у колонама алгоритама су количници битови по бајту (што мање - то боље), а  $H(X_2|X_1) = -\sum_{a,b} p(a,b) \log_2 \frac{p(a,b)}{p(a)}$  је емпиријска условна ентропија сљедећег бајта у односу на претходни ( $p(a,b)$  је релативна учесталост пара сусједних бајтова, а  $p(a)$  прва компонента).

Из табеле се виде сљедећа запажања.

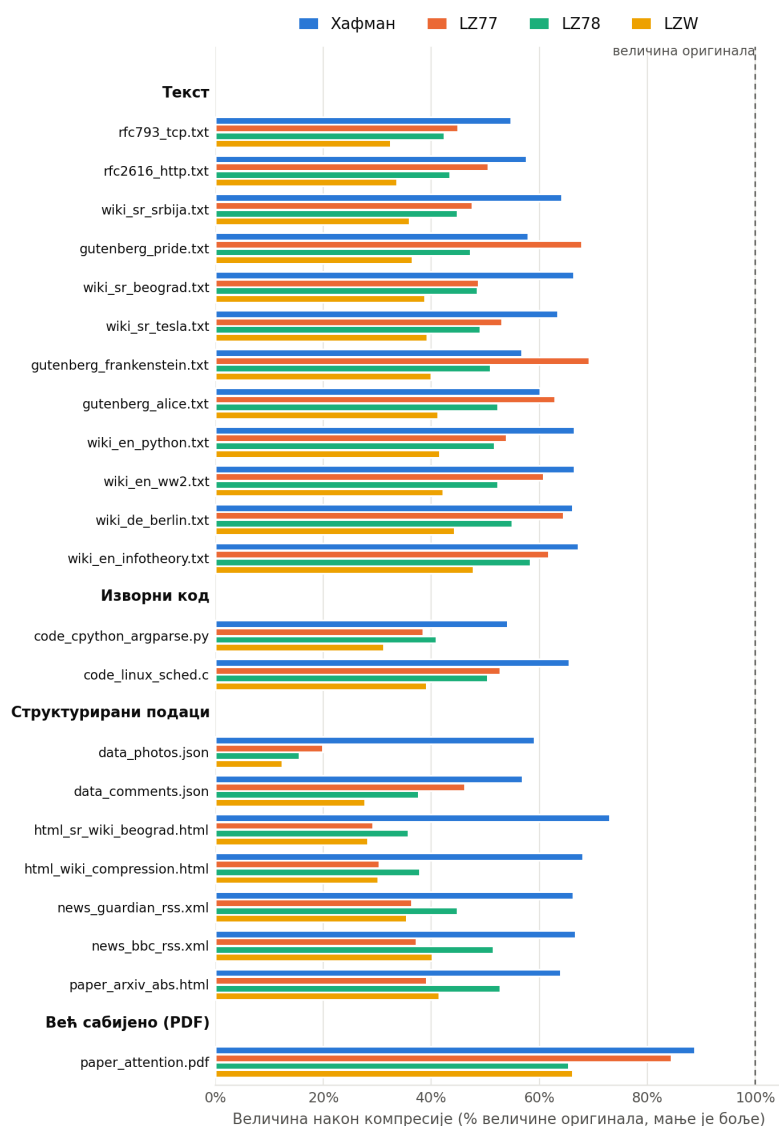
- Хафманов код никада није испод  $H(X)$  и увијек је близу ње, што је у складу са границама  $H(X) \leq E[L] < H(X) + 1$ . На већим текстовима је 0.04–0.09 бита изнад, а на најмањем 0.3, јер тамо таблица фреквенција у заглављу чини већи дио излаза.
- На тексту све три рјечничке методе иду *испод*  $H(X)$ , што не противријечи Шеноновој теореме: њена граница важи за независне симболе, а слова у тексту то нису. Условна ентропија  $H(X_2|X_1)$  показује колико је зависност велика (за енглески текст 5.29 према 3.84; код ћирилице је дио зависности последица UTF-8 записа са два бајта по слову).
- Случајни бајтови се не могу сажети: Хафман остаје на 8.01, а LZ методе *повећавају* датотеку, LZ77 чак на 170% (тројка је 28 бита и кад нема поклапања).

## 4.3 Већи скуп улаза

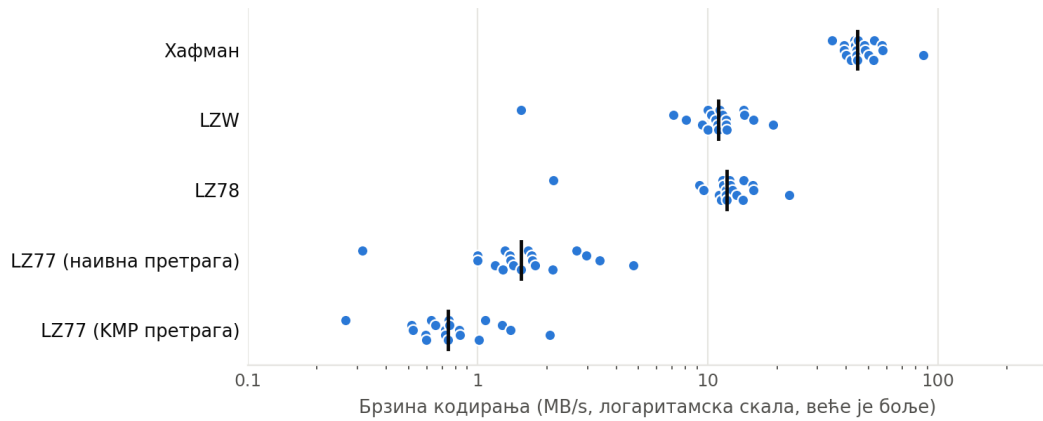
Да закључци не би зависили од неколико изабраних примјера, поновили смо поређење на 22 датотеке преузете са интернета: чланци са Википедије (на српском, енглеском и њемачком), књиге, RFC документи, изворни код, JSON, RSS и HTML странице и један PDF. Сваку смо кодирали свим алгоритмима и декодирали назад.

На првом графикону се види да је LZW готово свуда најбољи, LZ78 му је обично најближи, а LZ77 и Хафман су слабији. Хафман је најслабији на структурираним подацима (JSON, XML, HTML): ту се стално понављају исте ознаке и кључеви, а он их не види јер гледа само појединачне бајтове, док рјечничке методе управо на томе добијају. На обичном тексту су разлике мање, а на књигама је LZ77 чак лошији од Хафмана: у прози има мало дугих понављања унутар малог прозора, а његова тројка је скупа и за кратка поклапања. На двије најмање датотеке LZ77 је бољи од LZW-а, вјероватно зато што LZW-у треба више података да напуни рјечник. Такође, значајно је напоменути да комбинација LZ77 и Хафмановог кодирања постиже најбољи резултат од свих.

На другом графикону је приказана брзина кодирања. Хафман је најбржи, затим долазе LZ78 и LZW (сличне брзине), а LZ77 је најспорији, јер за сваку позицију претражује прозор. Код LZ77 је на обичним подацима наивна претрага бржа од КМР-а: поређење већином не успије већ на првом знаку, док КМР сваки пут прође цио прозор. КМР је бржи тек на јако понављајућим подацима (у нашим пробама слово а и периодичан текст), гдје су поклапања дуга.



*Величина након компресије као проценат оригинала. Датотеке су груписане по врсти и поређане по резултату LZW алгоритма.*



*Брзина кодирања за датотеке од 100 KB навише. Једна тачка је једна датотека, а вертикална црта је медијана. Мјерено је једном по датотеци.*

# Примјене

## 5.1 Употреба у пракси

Ови алгоритми се ријетко користе сами, већ као степени већих формата: Хафманов код и LZ77 заједно чине DEFLATE (gzip, ZIP, PNG), LZW је у формату GIF и алату `compress`, а Хафманов код је и завршни степен у формату JPEG и алату *bzip2*. Новији компресори (xz, zstd) су потомци LZ77 са бољим кодирањем излаза.

## 5.2 Стварне стопе компресије

Јавно поређење реалних компресора даје *Large Text Compression Benchmark* (Мет Махони).<sup>1</sup> Он мјери величину компресованог текста *enwik8*, првих  $10^8$  бајтова енглеске Википедије (XML). Ево резултата за алате који користе алгоритме из овог рада:

---

<sup>1</sup>[mattmahoney.net/dc/text.html](http://mattmahoney.net/dc/text.html)

| програм (опције)   | алгоритам       | величина (В) | % оригинала | бит/бајт |
|--------------------|-----------------|--------------|-------------|----------|
| compress 4.3d      | LZW             | 45 763 941   | 45.8        | 3.66     |
| gzip 1.3.5 (-9)    | LZ77 + Хафман   | 36 445 248   | 36.4        | 2.92     |
| bzip2 1.0.2 (-9)   | BWT + Хафман    | 29 008 736   | 29.0        | 2.32     |
| zstd 0.6.0 (-22)   | LZ77 + кодирање | 25 405 601   | 25.4        | 2.03     |
| xz 5.2.1 (LZMA2)   | LZ77 + кодирање | 24 703 772   | 24.7        | 1.98     |
| 7-Zip 4.46a (PPMd) | PPM             | 21 197 559   | 21.2        | 1.70     |

Овај редослијед је супротан ономе из наших мјерења, гдје је LZW био бољи од LZ77. Разлог је то што gzip изнад LZ77 користи Хафманов код (тј. DEFLATE), док **compress** ограничава величину рјечника (подразумијевано на 16-битне кодове).

# Закључак

Рад је почео питањем колико се подаци могу сажети, а одговор је дала Шенонова теорема: за независне симболе доња граница је ентропија. Хафманов код је најбољи начин да јој се приближимо кодирањем симбол по симбол, и експерименти показују да јој се на већим улазима приближавамо на мање од 0.1 бита. Рјечничке методе користе оно што теорема не обухвата, зависност међу симболима, и зато на компресији текста њихови параметри достижу вриједности мање од ентропије.

Међу рјечничким методама у нашим мјерењима најбољи је LZW (на скоро свим датотекама). Када се LZ77 споји са Хафмановим кодом, добија се DEFLATE, који се у пракси показује знатно бољим од ових метода посматраних појединачно.

Имплементације су упрошћене: формат датотеке је наш, а резултати су мјерени на малом броју улаза и на једном рачунару.